DOCUMENT D'ARCHITECTURE · V1.2

Architecture et mise en production ALTERRA

Suivi du pointage journalier et de la paie à la tâche pour la main-d'œuvre casuelle (MOC) sur les sites ALTERRA. Ce document décrit l'architecture cible et donne la marche à suivre complète pour provisionner et mettre en service le serveur de production, de façon autonome.

Hébergement : on-premise, Antananarivo Exposition : Cloudflare Tunnel

Monorepo : backend · admin · pwa · infra

01 Vue d'ensemble

Trois applications composent le système, servies par une seule API. Pas de microservices : un monolithe modulaire suffit au périmètre (5 sites, quelques centaines de MOC, une à deux dizaines d'utilisateurs simultanés). La vraie difficulté n'est pas le volume, c'est la connectivité terrain intermittente, d'où le choix d'une PWA offline-first.

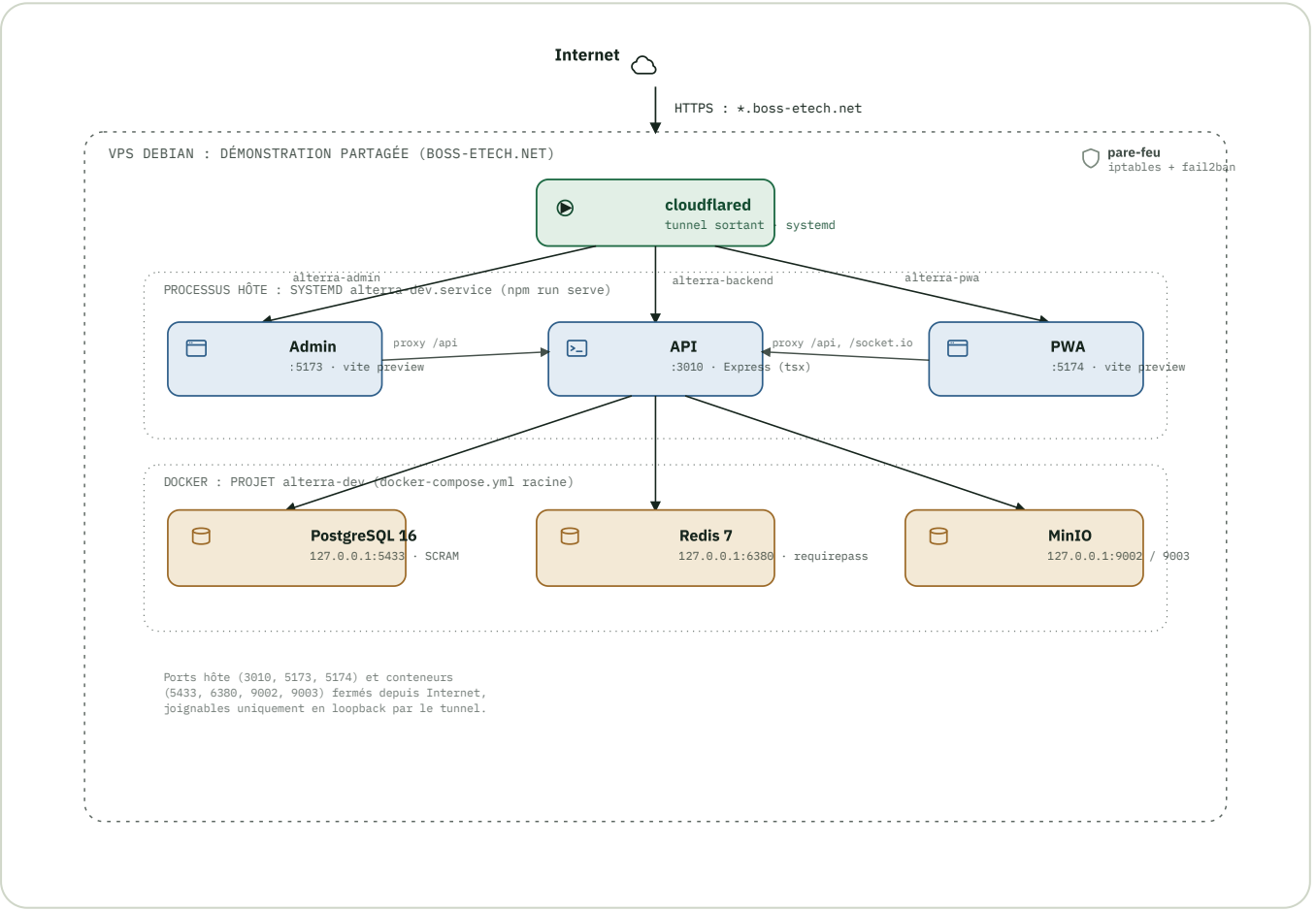
APPLICATION	RÔLE	UTILISATEURS
`pwa/` PWA terrain	Saisie du pointage sur site, offline-first (IndexedDB/Dexie), synchronisation par lots dès que le réseau revient	Chefs d'équipe, chefs de service
`admin/` Back-office	Référentiels (sites, zones, activités, travailleurs), validation, paie et réconciliation MVola, rapports, audit	Chefs de service, administrateurs
`backend/` API	Cœur métier unique : authentification, RBAC, synchronisation, calcul de paie, biométrie, rapports	Consommée par les deux fronts

ÉTAT RÉEL AUJOURD'HUI

Le seul environnement en ligne actuellement est la **démonstration** (§02) : un VPS partagé, exploité en mode « build » derrière un tunnel Cloudflare. **Aucun serveur de production n'existe à ce jour.** La section §06 n'est pas une description de l'existant mais la marche à suivre pour en construire un, à partir de la configuration déjà présente dans le dépôt (`infra/docker-compose.prod.yml` et scripts associés), rien de ce qu'elle décrit n'est déployé tant que ces étapes n'ont pas été exécutées.

02 Architecture actuelle

Ce schéma représente exactement ce qui tourne aujourd'hui : l'environnement de démonstration, seul environnement en ligne à ce jour. Rien n'y figure qui ne soit vérifiable dans `infra/systemd/alterra-dev.service`, `infra/cloudflared/config.dev.yml` et `docker-compose.yml`.



Environnement de démonstration, seul en ligne aujourd'hui : un VPS Debian partagé avec d'autres projets. Le tunnel Cloudflare route trois noms d'hôte vers trois ports locaux ; l'admin et la PWA (builds Vite) proxifient elles-mêmes `/api` vers l'API ; PostgreSQL, Redis et MinIO tournent en conteneurs Docker publiés en loopback uniquement.

Bordure réseau (tunnel) — Processus applicatifs — Données (conteneurs Docker)

SUJET	ÉTAT RÉEL AUJOURD'HUI
Service applicatif	`alterra-dev.service` (systemd) → `npm run serve` : API en `tsx` (sans watch) + `vite preview` pour Admin et PWA, journal dans `/home/debian/alterra-dev.log`
Exposition	Tunnel Cloudflare sortant uniquement (`cloudflared.service`), trois noms d'hôte sous `boss-etech.net` , aucune IP fixe ni port entrant côté ALTERRA
Reverse proxy	Aucun : Admin et PWA proxifient `/api` (et `/socket.io` pour la PWA) directement dans leur propre serveur `vite preview`
Infrastructure de données	Conteneurs Docker du projet `alterra-dev` (`docker-compose.yml` racine) : PostgreSQL 16, Redis 7, MinIO, tous publiés sur `127.0.0.1` uniquement
Sauvegarde / supervision	Aucune automatisée sur cet environnement : pas de pgBackRest, pas de copie hors site, pas de sonde Uptime Kuma/Netdata (voir §06 pour la cible)

03 Stack technique

Choix technologiques du dépôt. La colonne **statut** distingue ce qui tourne réellement aujourd'hui de ce qui n'est que configuré dans le dépôt, prêt à être déployé (voir §06).

COUCHE	TECHNOLOGIE	STATUT
Langage	TypeScript strict : backend, admin, pwa	**en service**
API	Node.js 22 LTS, Express 4, Prisma 5 : monolithe modulaire, ESM	**en service**
Validation	Zod : schéma par route	**en service**
Authentification	JWT (accès 15 min + refresh 7 j en cookie httpOnly), argon2id, TOTP (otplib)	**en service**
Base de données	PostgreSQL 16 : enums natifs, jsonb, index composites	**en service**
Cache / files	Redis 7 + BullMQ + ioredis	**en service**
Stockage objet	MinIO (compatible S3) : photos MOC, rapports, icône applicative	**en service**
Documents	ExcelJS, Handlebars + Puppeteer (PDF)	**en service**
Front admin	React 18 + Vite, Radix UI + Tailwind	**en service**
Front PWA	React 18 + Vite PWA, Dexie (IndexedDB), Service Worker (Workbox)	**en service**
Exposition	Cloudflare Tunnel (cloudflared)	**en service**
CI/CD	GitHub Actions : lint, tests, build d'image, déploiement à approbation manuelle	**en service**
Reverse proxy	Nginx 1.26 : TLS 1.3, HTTP/2	**prêt (prod), pas déployé**
Sauvegarde	pgBackRest (PITR) + rclone crypt → Backblaze B2 : stratégie 3-2-1	**prêt (prod), pas déployé**

COUCHE	TECHNOLOGIE	STATUT
Supervision	Uptime Kuma + Netdata + sonde externe	**prêt (prod), pas déployé**

04 Modèle de données

Schéma Prisma (`backend/prisma/schema.prisma`) : entités principales :

ENTITÉ	RÔLE
`User`	Comptes : rôle ADMIN / CHEF_SERVICE / CHEF_EQUIPE, rattaché à un site et/ou une équipe
`Site` / `Zone` / `Parcelle`	Lieu d'intervention et découpage géographique
`Team`	Équipe de terrain rattachée à un site
`Worker`	MOC : matricule, numéro MVola, statut, photo
`ActivityCategory` / `ActivitySubActivity` / `Unit`	Référentiel des tâches facturables, tarifs versionnés
`Pointage`	Ligne de pointage : quantité × tarif figé, idempotent via `clientUuid`
`Badge` / `PresenceRecord`	Carte NFC associée à un travailleur, pointage de présence
`Payment`	Paie calculée par cycle, réconciliation MVola
`ActivityRequest` / `WorkerRequest` / `ClarificationRequest`	Workflows de demande et validation
`AuditLog`	Journal d'audit générique (avant/après)

Deux garanties structurent la synchronisation terrain : **idempotence** (`clientUuid` généré côté PWA, rejoué sans doublon) et **tarif figé** (`unitRateSnapshot` calculé à la saisie : un changement de tarif n'affecte jamais les pointages déjà enregistrés).

05 Sécurité

Authentification

JWT d'accès 15 min (HS256) + refresh 7 j en cookie `httpOnly`, `sameSite=strict`, rotatif (liste noire Redis à la rotation). Mots de passe argon2id. TOTP demandé dès qu'un ADMIN est enrôlé.

Anti-bruteforce

Rate limit 10 req / 5 min sur `/api/v1/auth` en production, verrouillage par e-mail (10 échecs / 15 min, comptes inconnus inclus). Vérification factice sur compte inconnu : pas d'énumération par le temps de réponse.

Autorisation (RBAC + RLS)

Trois rôles appliqués par route. Cloisonnement site/équipe en plus : une extension Prisma injecte automatiquement le filtre `siteId`/`teamId` sur les modèles sensibles, en lecture et en écriture.

Secrets

Au démarrage, l'API refuse de tourner en production si un secret vaut encore sa valeur d'exemple publique. Rotation via `infra/scripts/rotate-secrets.sh` et `rotate-infra-secrets.sh`.

Fichiers

Buckets MinIO privés, URLs pré-signées de 15 min. Aucun fichier servi directement par Express, sauf l'icône applicative, relayée avec `Content-Security-Policy: sandbox` et `nosniff`.

Transport & exposition

TLS 1.3 partout, HSTS. Aucun service de données exposé hors du réseau Docker interne. Cloudflare Tunnel : connexion sortante uniquement, zéro port entrant côté ALTERRA.

RÈGLE NON NÉGOCIABLE

Un serveur de développement (`vite`, `tsx watch`) ne doit **jamais** être placé derrière une exposition publique. Il sert les fichiers sources du projet et a déjà été exploité en septembre 2026 sur l'environnement de démonstration. En production comme en démo, seuls des **builds** servis en statique (nginx / `vite preview`) sont exposés.

06 Mise en production : marche à suivre

Séquence complète pour provisionner un nouveau serveur de production et y déployer ALTERRA, de façon autonome. Chaque étape renvoie aux scripts réels du dépôt (`infra/scripts/`) : aucune commande ci-dessous n'est approximative.

RIEN N'EST DÉPLOYÉ À CE STADE

Cette section décrit une cible à construire sur un **nouveau serveur**, distinct du VPS de démonstration (§02). La configuration (`infra/docker-compose.prod.yml`, scripts, nginx, pgBackRest) existe dans le dépôt et a été revue, mais aucun hôte ne l'exécute encore.

Choix retenus pour la cible production, et leur raison

SUJET	CHOIX	POURQUOI
Hébergement	Serveur on-premise, Antananarivo	Souveraineté des données, conformité loi malgache n° 2014-038
Exposition	Cloudflare Tunnel (sortant uniquement)	Pas d'IP fixe nécessaire, aucun port entrant, TLS et anti-DDoS inclus
Conteneurisation	Docker Compose	Reconstructible depuis Git seul ; chaque service remplaçable indépendamment
Base de données	PostgreSQL 16, volume dédié	WAL archivé pour restauration à un point dans le temps (PITR)
Fichiers	MinIO + URLs pré-signées (15 min)	Aucun fichier servi directement par Express ; contrôle d'accès par rôle
Résilience	Onduleur + bascule 4G	Le délestage électrique est un évènement normal à Antananarivo ; la PWA fonctionne hors ligne pendant l'indisponibilité

6.1 Prérequis

ÉLÉMENT	DÉTAIL
Serveur	Debian 12+ (on-premise ou VPS), accès root ou sudo, 2 vCPU / 4 Go RAM minimum recommandés
Domaines	`app.alterra.mg` (PWA) et `admin.alterra.mg` (back-office), DNS gérés chez Cloudflare
Exposition	Compte Cloudflare avec un tunnel (recommandé), ou à défaut une IP fixe + ouverture du port 443
Sauvegarde hors site	Compte Backblaze B2 + configuration `rclone crypt`
Supervision	URL de ping Healthchecks.io pour surveiller le job de sauvegarde
Dépôt Git	Accès en lecture au dépôt ALTERRA (clé SSH de déploiement)
Registre d'images	Accès à GHCR (GitHub Container Registry) où la CI publie les images `api`, `admin`, `pwa`

6.2 Provisionnement initial

Lancer le script de provisionnement en root sur le serveur fraîchement installé, il installe Docker, crée l'utilisateur de déploiement et ouvre le pare-feu minimal (SSH, 80, 443) :

```
# sur le serveur, en root
sudo REPO_URL=git@github.com:ORG/alterra.git ./infra/scripts/setup-vps.sh
```

Le script affiche ensuite les étapes manuelles restantes, reprises en détail ci-dessous :

```
# 1. Cloner le dépôt dans /opt/alterra
sudo -u deploy git clone git@github.com:ORG/alterra.git /opt/alterra
cd /opt/alterra
```

6.3 Secrets et variables d'environnement

1 Copier et remplir `.env.prod`

```
cp .env.prod.example .env.prod
# éditer .env.prod : voir la référence complète en §08
```

Toute valeur `CHANGE_ME` doit être remplacée avant le premier démarrage, l'API refuse de démarrer en production sinon.

2 Créer les secrets Docker

```
mkdir -p infra/secrets
# mot de passe PostgreSQL
openssl rand -base64 32 > infra/secrets/pg_password.txt
# identifiants MinIO
echo alterra_prod_admin > infra/secrets/minio_user.txt
openssl rand -base64 32 > infra/secrets/minio_pass.txt
chmod 600 infra/secrets/*.txt
```

Ces fichiers sont montés en secrets Docker (jamais en variable d'environnement en clair), voir `infra/docker-compose.prod.yml`. `infra/secrets/` est exclu de Git.

6.4 Tunnel Cloudflare

Authentification et création du tunnel, une seule fois, depuis le serveur :

```
cloudflared tunnel login
cloudflared tunnel create alterra-prod
cloudflared tunnel route dns alterra-prod app.alterra.mg
cloudflared tunnel route dns alterra-prod admin.alterra.mg
```

Le fichier d'identifiants généré (`~/.cloudflared/<tunnel-id>.json`) doit être copié vers `infra/secrets/cloudflared/alterra-prod.json`, le chemin monté par le conteneur `cloudflared` (voir `infra/cloudflared/config.yml`, déjà versionné et prêt à l'emploi).

6.5 Certificats TLS

```
cd /opt/alterra/infra
CERTBOT_EMAIL=ops@alterra.mg ./scripts/certbot-init.sh
```

Le script arrête nginx le temps d'obtenir les certificats Let's Encrypt (mode `standalone`) pour les deux domaines, puis le redémarre. Le renouvellement automatique est mis en place à l'étape 6.8.

6.6 Premier déploiement

```
export GHCR_IMAGE_PREFIX=ghcr.io/VOTRE_ORG/alterra
export TAG=<git-sha>
./scripts/deploy.sh
```

`deploy.sh` enchaîne automatiquement :

1. Sauvegarde pré-déploiement (`pg_dump`, ignorée au tout premier lancement)
2. `docker compose pull` : récupère les images taguées depuis GHCR
3. `npx prisma migrate deploy` : applique les migrations de schéma
4. `docker compose up -d` : démarre tous les services
5. Test de fumée automatique (`smoke-test.sh`)

6.7 Vérification

```
./scripts/smoke-test.sh
SMOKE_APP_URL=https://app.alterra.mg SMOKE_ADMIN_URL=https://admin.alterra.mg
./scripts/smoke-test.sh
```

Vérifie successivement : API (`/health`), PostgreSQL (`pg_isready`), Redis (`PING`), MinIO, puis les fronts statiques admin et PWA, et enfin les deux URL publiques si fournies.

6.8 Sauvegardes et supervision automatiques

```
HEALTHCHECK_URL=https://hc-ping.com/<votre-uuid> ./scripts/install-cron.sh
```

Installe deux tâches cron sur l'hôte :

HEURE	TÂCHE	DÉTAIL
`02:00`	Sauvegarde hors site	`backup-offsite.sh` : dump PostgreSQL + buckets MinIO → chiffrement `rclone crypt` → Backblaze B2, puis ping du healthcheck
`03:00`	Renouvellement TLS	`certbot renew --quiet`, redémarrage nginx si un certificat a été renouvelé

Uptime Kuma et Netdata tournent déjà comme services du `docker-compose.prod.yml`, aucune étape supplémentaire. Uptime Kuma surveille `/health` sur les trois fronts ; prévoir en plus une sonde externe (UptimeRobot ou équivalent) pour détecter une panne du tunnel lui-même.

ABSENCE DE PING = ALERTE

Le job de sauvegarde notifie Healthchecks.io à chaque exécution réussie. C'est le **silence** qui signale un échec, pas un message d'erreur explicite, configurer l'alerte Healthchecks.io en conséquence dès l'installation.

07 Déploiements suivants & retour arrière

Nouveau déploiement

```
cd /opt/alterra/infra
TAG=<git-sha>
./scripts/deploy.sh
```

Backup pré-déploiement → pull → migration → bascule → test de fumée, dans cet ordre, à chaque exécution.

Rollback

```
TAG=<sha-précédent>
./scripts/rollback.sh
```

Ne revient **pas** sur les migrations Prisma appliquées après le tag cible. Si le rollback franchit une migration cassante, restaurer depuis la sauvegarde (`restore.sh`) plutôt que de revenir en arrière.

Le pipeline CI/CD (`.github/workflows/deploy.yml`) automatise tout cela depuis un `git push` sur `main` : tests → build et publication des images sur GHCR → déploiement SSH sur le serveur, derrière une **approbation manuelle** (environnement GitHub `production`). Secrets du dépôt à configurer : `PROD_SSH_HOST`, `PROD_SSH_USER`, `PROD_SSH_PRIVATE_KEY`.

08 Variables d'environnement

Référence complète de `.env.prod.example`. Toute valeur marquée `CHANGE_ME` est un gabarit à remplacer impérativement.

VARIABLE	EXEMPLE / DÉFAUT	RÔLE
`NODE_ENV`	`production`	Active les vérifications strictes (secrets, cookies `secure`)
`PORT`	`3001`	Port interne de l'API (derrière nginx, jamais exposé directement)
`LOG_LEVEL`	`info`	Niveau de log pino
`DATABASE_URL`	`postgresql://alterra:CHANGE_ME@postgres:5432/alterra`	Connexion Prisma vers PostgreSQL
`REDIS_URL`	`redis://redis:6379`	Connexion Redis (rate-limit, jobs, sessions)
`MINIO_ENDPOINT` / `MINIO_PORT` / `MINIO_USE_SSL`	`minio` / `9000` / `false`	Adresse interne du conteneur MinIO
`MINIO_ACCESS_KEY` / `MINIO_SECRET_KEY`	`CHANGE_ME`	Identifiants MinIO : doivent correspondre aux secrets Docker
`MINIO_BUCKET_PHOTOS` / `MINIO_BUCKET_REPORTS`	`photos-pointages` / `rapports-pdf`	Noms des buckets applicatifs
`JWT_SECRET` / `JWT_REFRESH_SECRET`	`CHANGE_ME_256_BITS_MINIMUM`	Signature des jetons : 256 bits minimum, jamais la valeur d'exemple
`JWT_ACCESS_TTL` / `JWT_REFRESH_TTL`	`15m` / `7d`	Durées de vie des jetons
`MFA_ENCRYPTION_KEY`	`CHANGE_ME` (64 car. hex)	Chiffrement des secrets TOTP en base
`ADMIN_ORIGIN` / `PWA_ORIGIN`	`https://admin.alterra.mg` / `https://app.alterra.mg`	Origines autorisées CORS
`MAX_UPLOAD_SIZE_MB`	`5`	Taille maximale des fichiers envoyés (photos)

VARIABLE	EXEMPLE / DÉFAUT	RÔLE
`BIOMETRIC_PROVIDER`	`MANUAL`	`MANUAL` / `MOCK` / `AXIAN` / `LOCAL_OFFLINE` : provider de vérification faciale
`AXIAN_API_URL` / `AXIAN_API_KEY`	n/a	Si `BIOMETRIC_PROVIDER=AXIAN` : endpoint et clé du prestataire
`AXIAN_MATCH_THRESHOLD`	`0.85`	Seuil de correspondance faciale accepté
`PDF_WORKER_ENABLED`	`true`	Active le worker BullMQ de génération de rapports
`MAILGUN_API_KEY` / `MAILGUN_DOMAIN` / `MAILGUN_FROM`	n/a	Envoi d'e-mails (réinitialisation de mot de passe)

Deux variables supplémentaires, lues par le cron hôte et non par le conteneur : HEALTHCHECK_URL (ping de sauvegarde) et RCLONE_REMOTE (cible rclone, défaut b2crypt:alterra-backups).

09 Exploitation courante

9.1 Gestes courants

```
# état des services
docker compose -f docker-compose.prod.yml ps
docker compose -f docker-compose.prod.yml logs -f api --tail=200

# redémarrer un service
docker compose -f docker-compose.prod.yml restart api
```

9.2 Restauration (test mensuel obligatoire)

```
COMPOSE_FILE=docker-compose.staging.yml ./scripts/restore.sh 2026-07-01
```

Procédure : arrêter les services applicatifs → lancer `restore.sh <date>` → vérifier les compteurs (sites, MOC) et la liste des buckets MinIO → redémarrer la stack → test de fumée. **Ne jamais restaurer en production** sans fenêtre de maintenance et validation ALTERRA, toujours répéter d'abord sur un environnement jetable.

9.3 Incidents fréquents

SYMPTÔME	DIAGNOSTIC
Coupure électrique	Normal, pas une exception. Onduleur ≈ 30 min. Les conteneurs redémarrent seuls (`restart: always`) ; vérifier que tout repasse `healthy` . La PWA continue de fonctionner hors ligne pendant la coupure.
Perte de connectivité	Bascule fibre → 4G automatique côté routeur. Le tunnel se rétablit sans reconfiguration dès le retour du réseau.
Silence du job de sauvegarde	Vérifier `docker compose logs backup-offsite` , l'espace disque (`df -h` , WAL et photos sont les postes à surveiller) et la connectivité vers Backblaze B2.
Certificat TLS proche expiration	Renouvellement automatique attendu. Si l'alerte persiste au-delà de J-7 : `certbot renew --force-renewal` puis redémarrer nginx.
Panne du tunnel Cloudflare	Sonde externe down, sonde interne (Uptime Kuma) OK. Vérifier `docker compose logs cloudflared` puis `restart cloudflared` .

9.4 Sinistre majeur (serveur détruit ou volé)

1. Provisionner un serveur de secours (répéter la §06 sur une nouvelle machine)
2. Cloner le dépôt Git : `infra/` est versionné et reconstruit le serveur intégralement
3. Restaurer la dernière sauvegarde hors site (`./scripts/restore.sh <dernière-date>`)
4. Rebasculer le tunnel Cloudflare vers le nouvel hôte (`cloudflared tunnel route dns`)
5. Vérifier `/health` sur les deux domaines avant de communiquer le retour de service

Pendant toute l'indisponibilité, la PWA terrain continue de fonctionner hors ligne et resynchronise au retour du service.

FENÊTRE DE MAINTENANCE

Mardi 12h–14h (créneau de faible activité terrain). Réservée aux mises à jour système et déploiements planifiés.

10 Risques connus

RISQUE	MITIGATION EN PLACE
Coupures électriques/réseau fréquentes	Onduleur, bascule 4G, mode hors ligne de la PWA
Sinistre serveur (panne, vol, incendie)	Sauvegardes 3-2-1 chiffrées, restauration testée mensuellement
Croissance au-delà de 5 sites	PostgreSQL/MinIO scalent verticalement ; architecture portable vers un serveur plus gros sans réécriture
Dépendance au prestataire biométrique externe	Repli `MANUAL` / `MOCK` / `LOCAL_OFFLINE` disponible
MFA non imposée à l'enrôlement ADMIN	Verrouillage par e-mail + rate limit réduisent le risque ; à durcir avant mise en production définitive (enrôlement obligatoire)
Comptes de démonstration aux mots de passe connus	Acceptée en environnement de démo uniquement. **Ne jamais créer ces comptes en production.**

ALTERRA : document vivant, à maintenir avec backend/prisma/schema.prisma, infra/*.yml et infra/scripts/

Sources : docs/architecture-technique.md · docs/deploy/production.md · docs/runbook.md